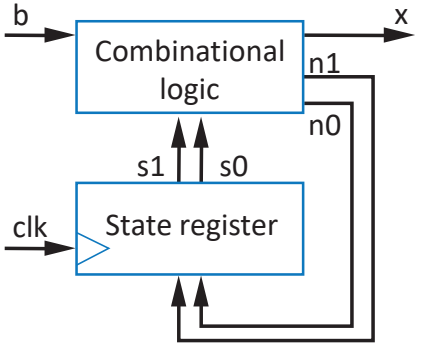
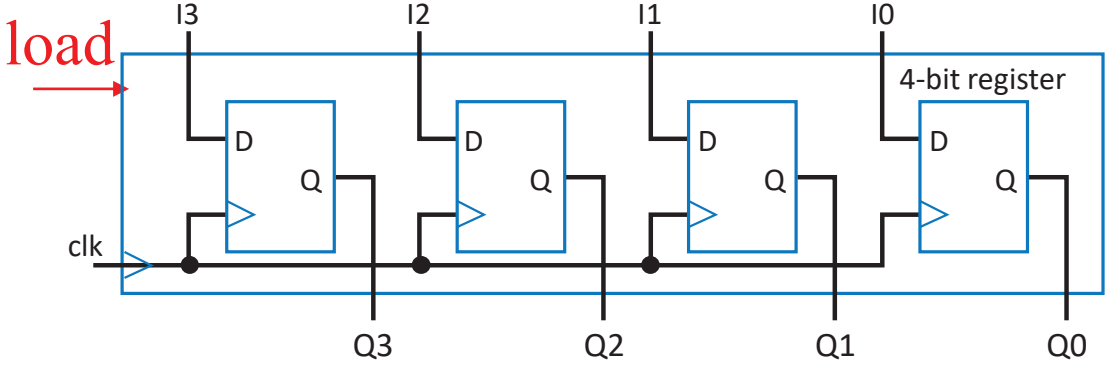
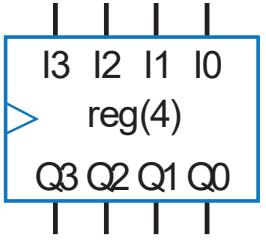
Registers

- *N-bit register*: Stores N bits, N is the *width*
  - Common widths: 8, 16, 32
  - Storing data into register: *Loading*
  - Opposite of storing: *Reading* (does not alter contents)
- Basic register of Ch 3: Loaded every cycle
  - Useful for implementing FSM—stores encoded state



a





Basic register loads on every clock cycle

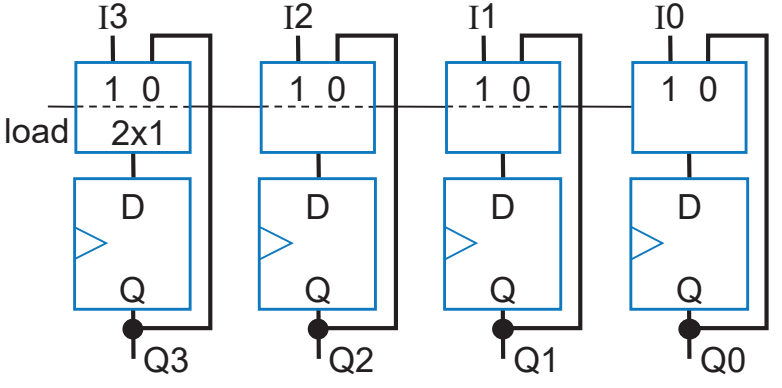
How extend to only load on certain cycles?

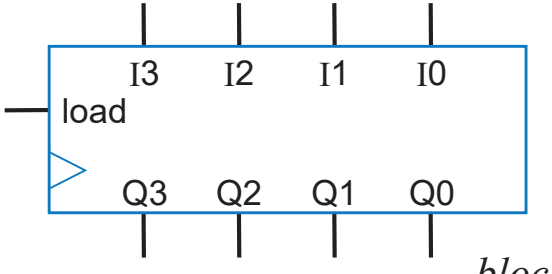


Digital Design 2e
Copyright © 2010
Frank Vahid

Register with Parallel Load

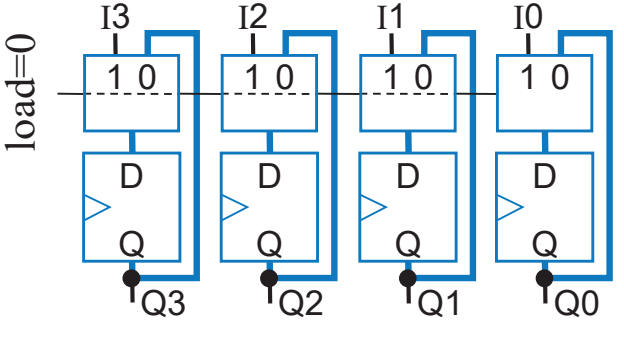
- Add 2x1 mux to front of each flip-flop
- Register's *load* input selects mux input to pass
  - load=0: existing flip-flop value; load=1: new input value

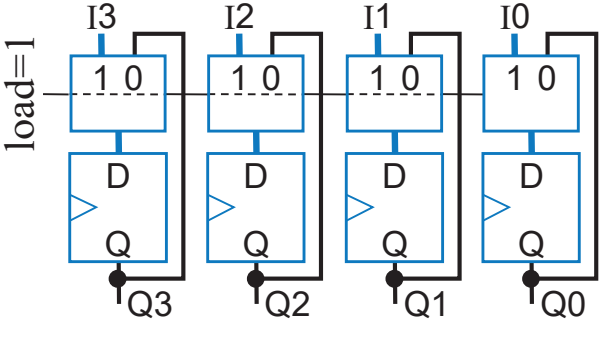




a

block symbol



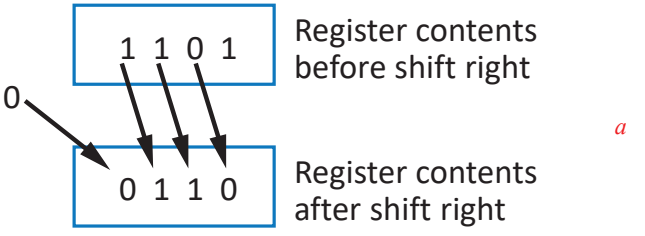


a

Digital Design 2e
Copyright © 2010
Frank Vahid

Shift Register

- Shift right
  - Move each bit one position right
  - Rightmost bit is “dropped”
  - Assume 0 shifted into leftmost bit



Q: Do four right shifts on 1001, showing value after each shift

A:

1001 (original)

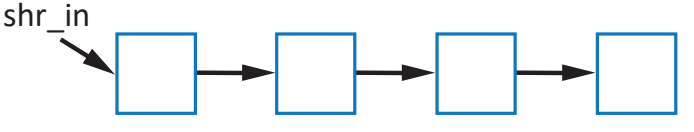
0100

0010

0001

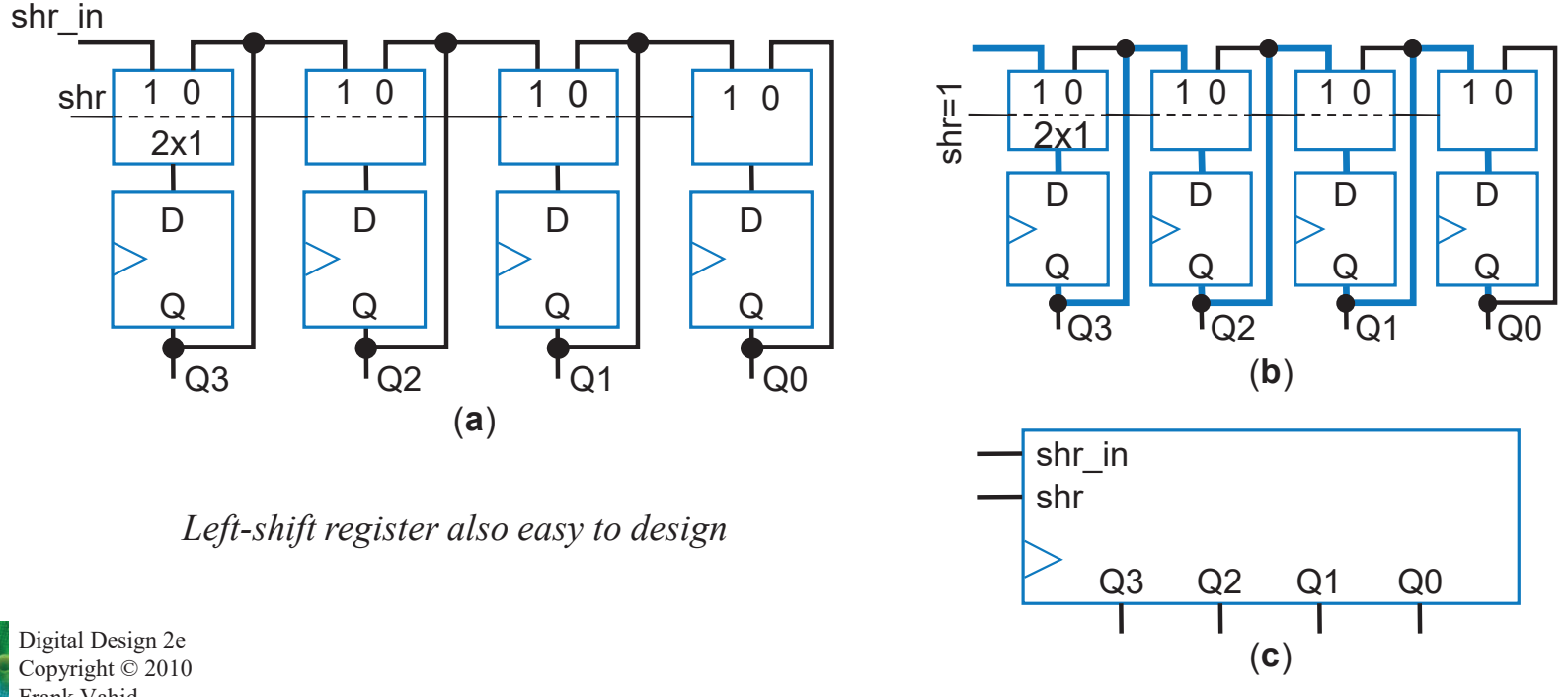
0000

- Implementation: Connect flip-flop output to next flip-flop’s input



Shift Register

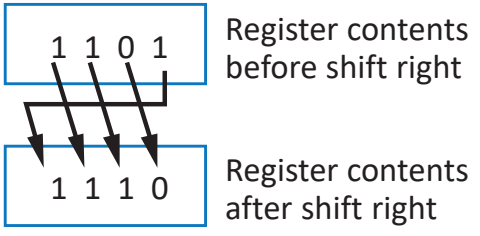
- To allow register to either shift or retain, use 2x1 muxes
  - shr: “0” means retain, “1” shift
  - shr_in: value to shift in
    - May be 0, or 1

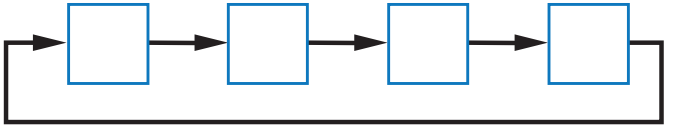


Left-shift register also easy to design

Rotate Register

- Rotate right: Like shift right, but leftmost bit comes from rightmost bit



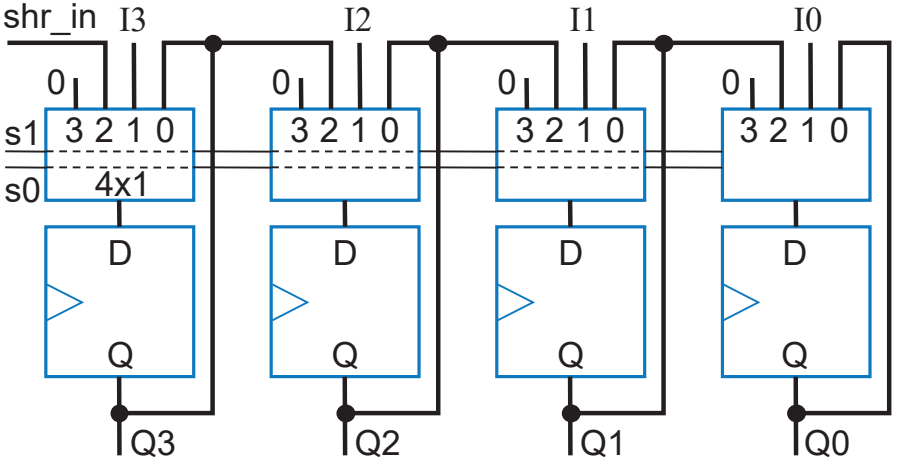


Multifunction Registers

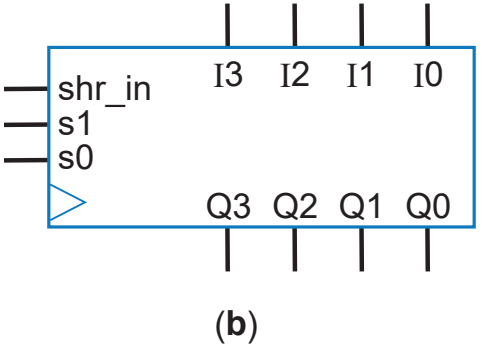
- Many registers have multiple functions
  - Load, shift, clear (load all 0s)
  - And retain present value, of course
- Easily designed using muxes
  - Just connect each mux input to achieve desired function

Functions:

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	(unused - let's load 0s)



(a)

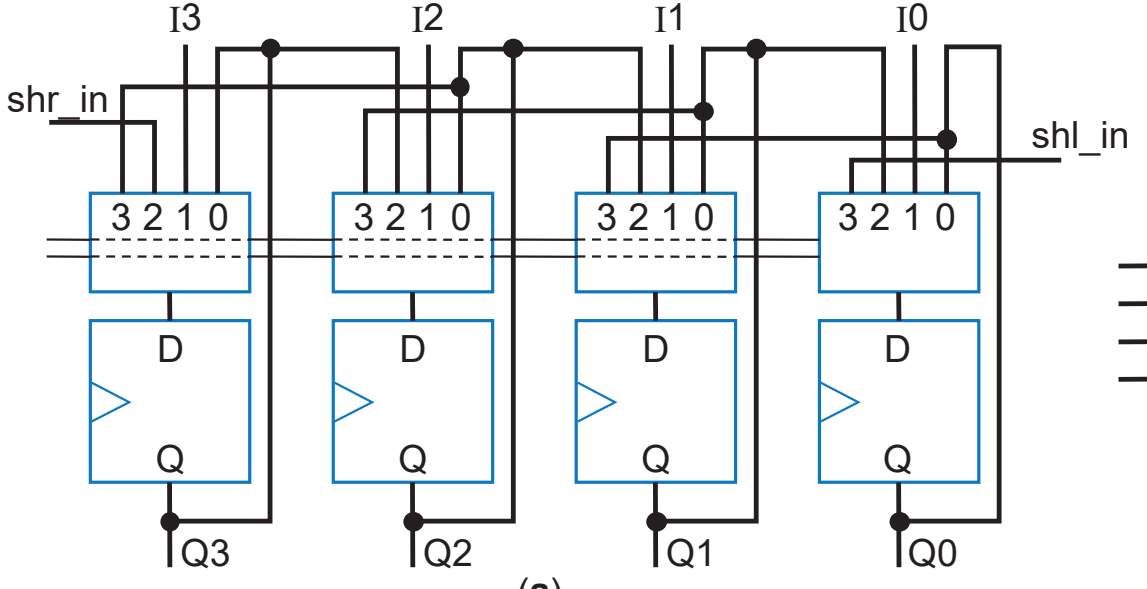


(b)

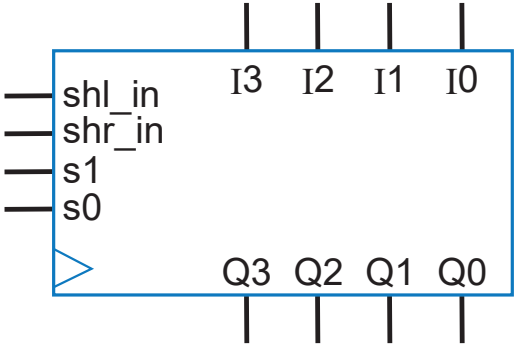
Digital Design 2e
Copyright © 2010
Frank Vahid

Multifunction Registers

s1	s0	Operation
0	0	Maintain present value
0	1	Parallel load
1	0	Shift right
1	1	Shift left



(a)

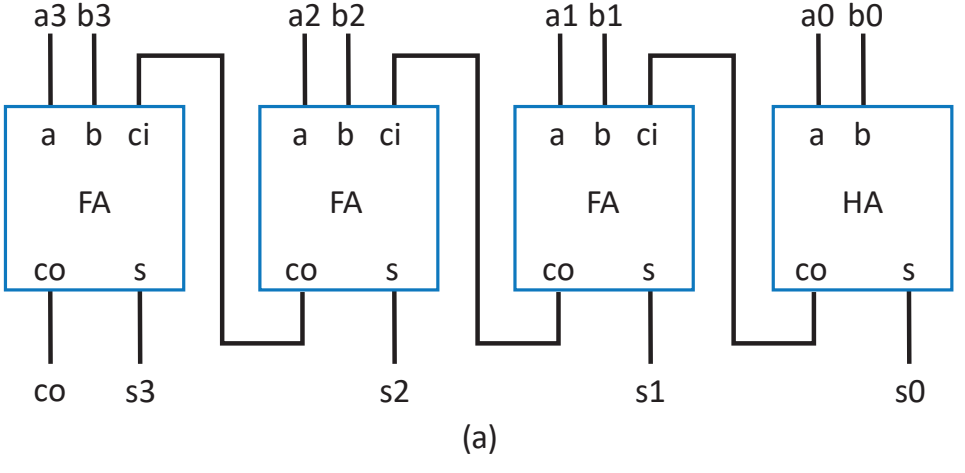


(b)

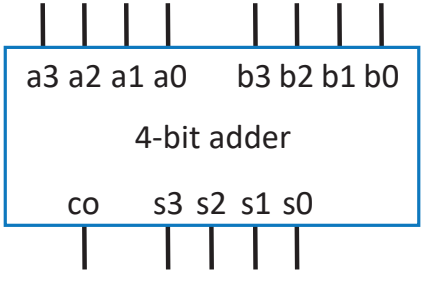
Copyright © 2010
Frank Vahid

Carry-Ripple Adder

- Using half-adder and full-adders, we can build adder that adds like we would by hand
- Called a ***carry-ripple adder***
  - 4-bit adder shown: Adds two 4-bit numbers, generates 5-bit output
    - 5-bit output can be considered 4-bit "sum" plus 1-bit "carry out"
  - Can easily build any size adder



(a)



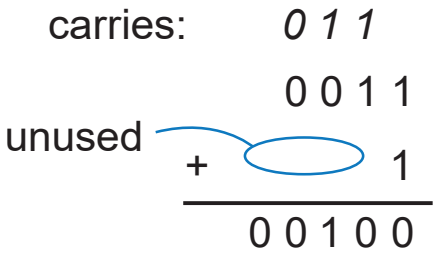
(b)

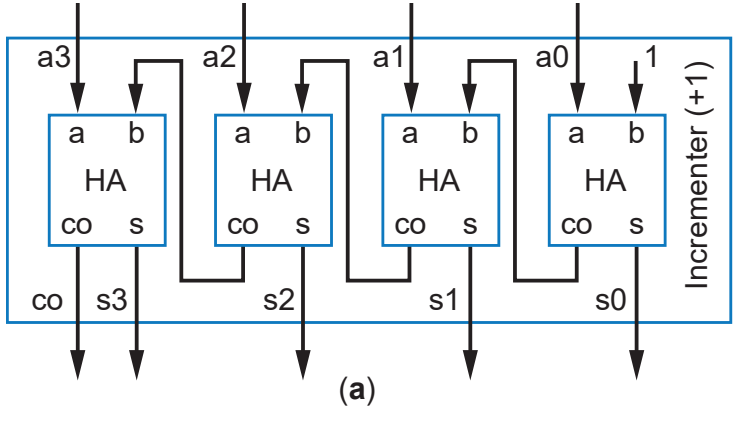
Copyright © 2010
Frank Vahid

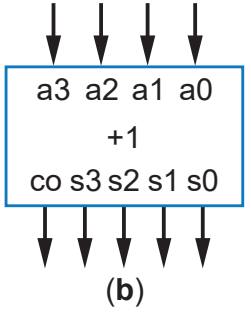
Incrementer

- Adds 1 to input A

Inputs				Outputs				
a3	a2	a1	a0	c0	s3	s2	s1	s0
0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1	0
0	0	1	0	0	0	0	1	1
0	0	1	1	0	0	1	0	0
0	1	0	0	0	0	1	0	1
0	1	0	1	0	0	1	1	0
0	1	1	0	0	0	1	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	1	0
1	0	1	0	0	1	0	1	1
1	0	1	1	0	1	1	0	0
1	1	0	0	0	1	1	0	1
1	1	0	1	0	1	1	1	0
1	1	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	0





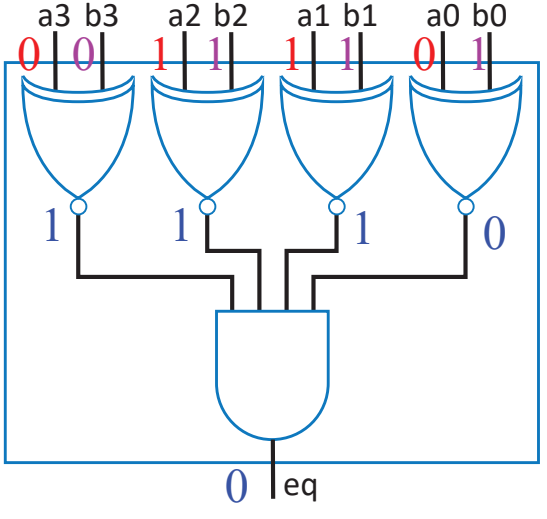


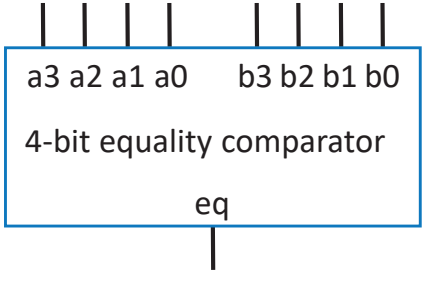
Could design using combinational design process, but smaller design uses carry-ripple, only need half-adders

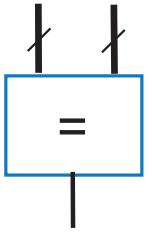
Comparators

- ***N-bit equality comparator***: Outputs 1 if two N-bit numbers are equal
  - 4-bit equality comparator with inputs A and B
    - a3 must equal b3, a2 = b2, a1 = b1, a0 = b0
      - Two bits are equal if both 1, or both 0
      - eq = (a3b3 + a3'b3') * (a2b2 + a2'b2') * (a1b1 + a1'b1') * (a0b0 + a0'b0')
    - Note that function inside parentheses is XNOR
      - eq = (a3 xnor b3) * (a2 xnor b2) * (a1 xnor b1) * (a0 xnor b0)

0110 = 0111 ?

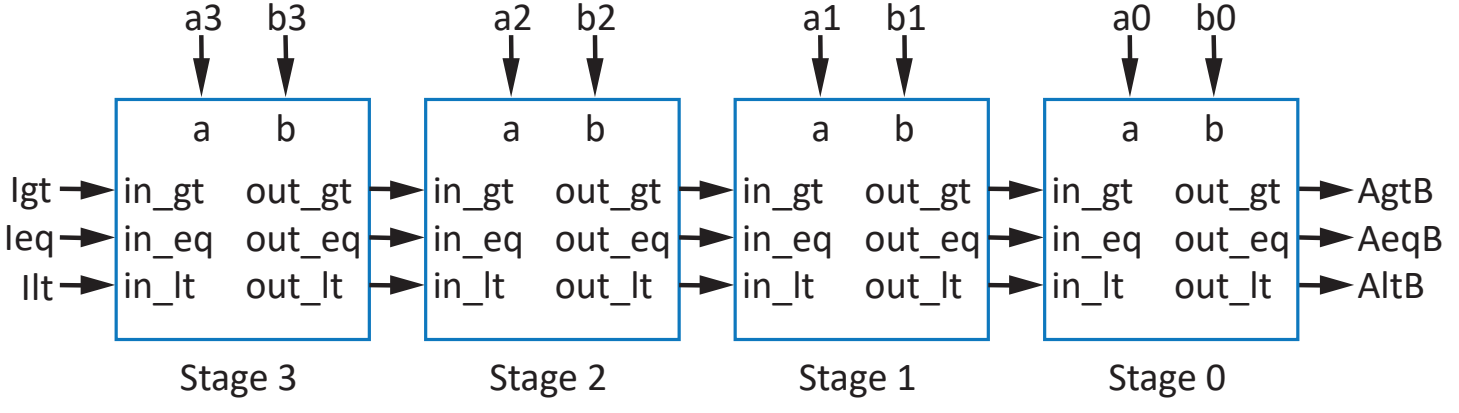




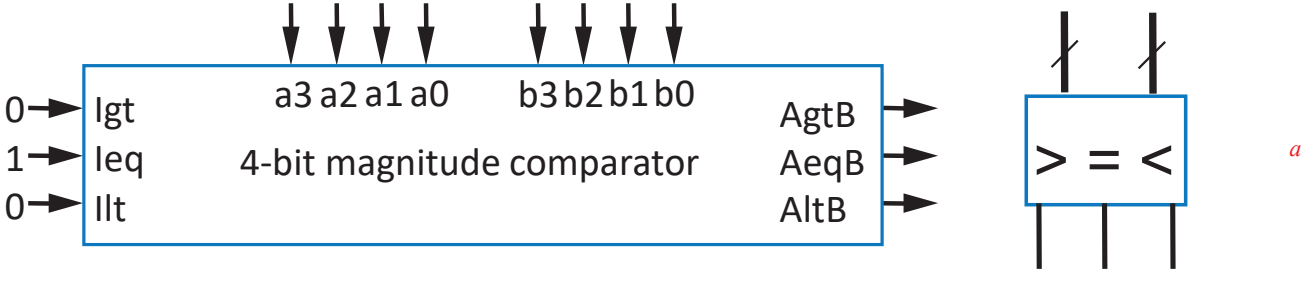


Magnitude Comparator

- By-hand example leads to idea for design
  - Start at left, compare each bit pair, pass results to the right
  - Each bit pair called a *stage*
  - Each stage has 3 inputs taking results of higher stage, outputs new results to lower stage

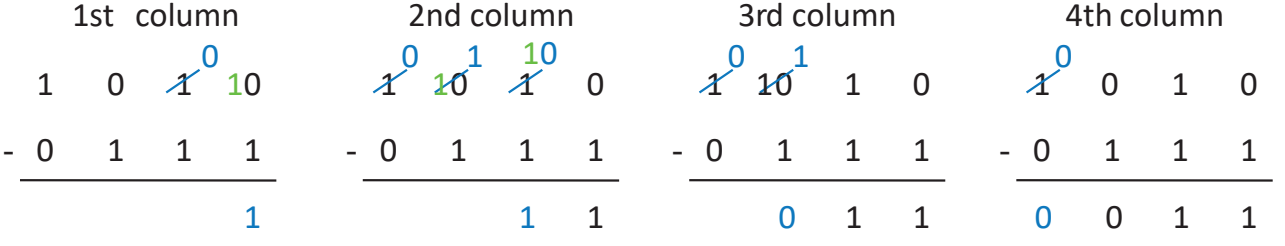


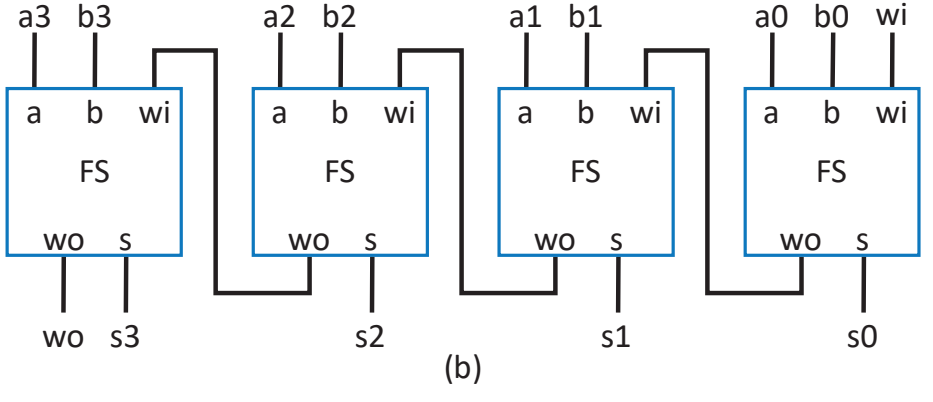
How design each stage?

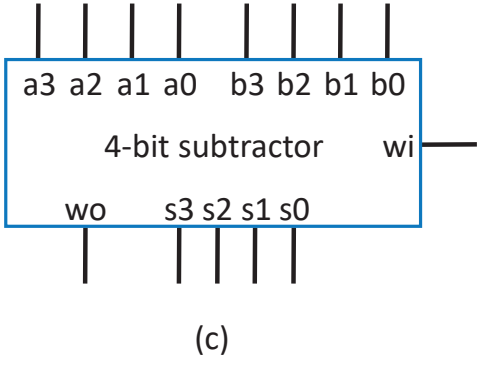


Subtractors and Signed Numbers

- Can build subtractor as we built carry-ripple adder
  - Mimic subtraction by hand
  - Compute the borrows from columns on left
    - Use full-subtractor component:
      - wi is borrow by column on right, wo borrow from column on left







Two's Complement Subtractor Built with an Adder

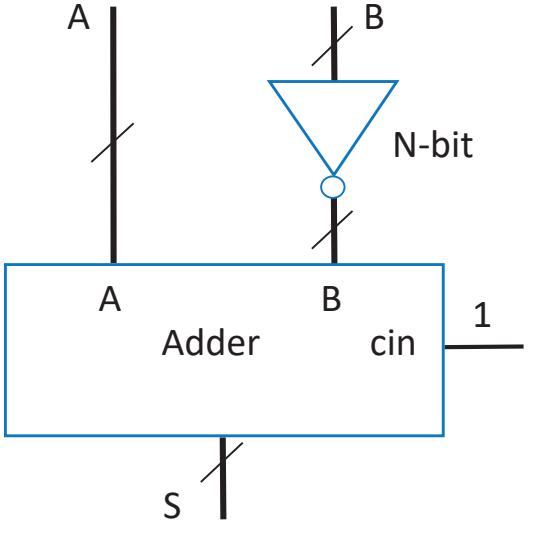
- Using two's complement

  A – B = A + (-B)

  = A + (two's complement of B)

  = A + invert_bits(B) + 1

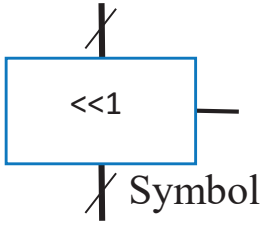
- So build subtractor using adder by inverting B's bits, and setting carry in to 1

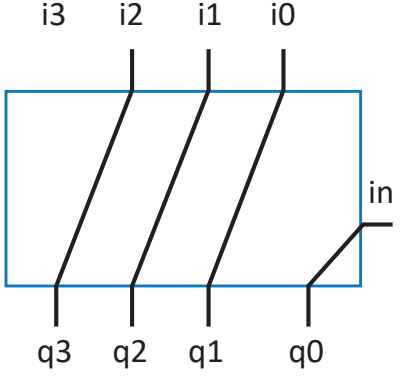


Copyright © 2010
Frank Vahid

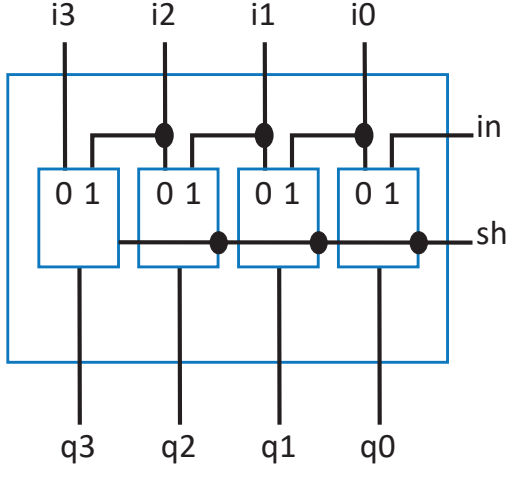
Shifters

- Shifting (e.g., left shifting 0011 yields 0110) useful for:
  - Manipulating bits
  - Converting serial data to parallel (remember earlier above-mirror display example with shift registers)
  - Multiply/divide by 2 (unsigned numbers only)
    - Shift left once is same as multiplying by 2. Ex: 0011 (3) becomes 0110 (6)
      - Why? Essentially appending a 0 -- Note that multiplying decimal number by 10 accomplished just be appending 0, i.e., by shifting left (55 becomes 550)
    - Shift right once same as dividing by 2

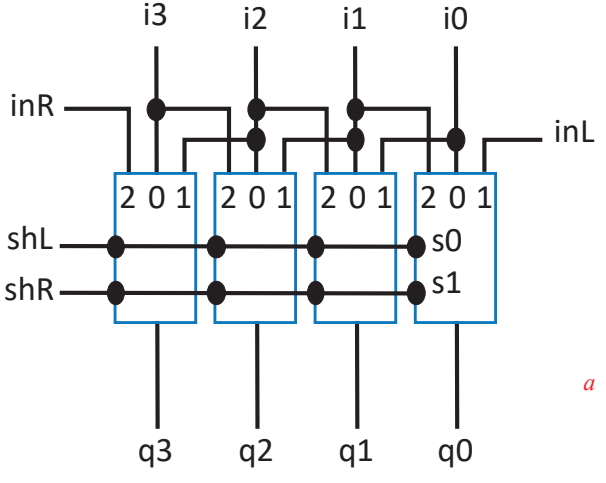




Left shifter



Shifter with left shift or no shift



Shifter with left shift, right shift, and no shift

Strength Reduction

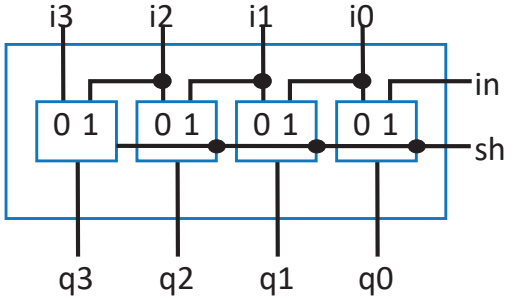
- Many multiplications not by power of 2 constants (2, 4, 8, …)
  - E.g., by 5, or 10
  - Could use multiplier component, but shifters are small and fast
- Replace multiplication by shifts and adds
  - Operator *strength reduction* (multiply is "stronger" than shift/add)
  - E.g., 5*C → 4*C + C (4*C same as C<<2)
- Replacing division by shifts and adds slightly harder
  - Approximate fraction using fraction having power of 2 denominator
  - E.g., C/5 = 0.20*C, approx. equals (102/512)*C = 0.199*C
    - (102/512)*C = C*(64+32+4+2)/512 = (C*64 + C*32 + C*4+C*2)/512
    - = ((C<<6) + (C<<5) + (C<<2) + (C<<1)) >> 9



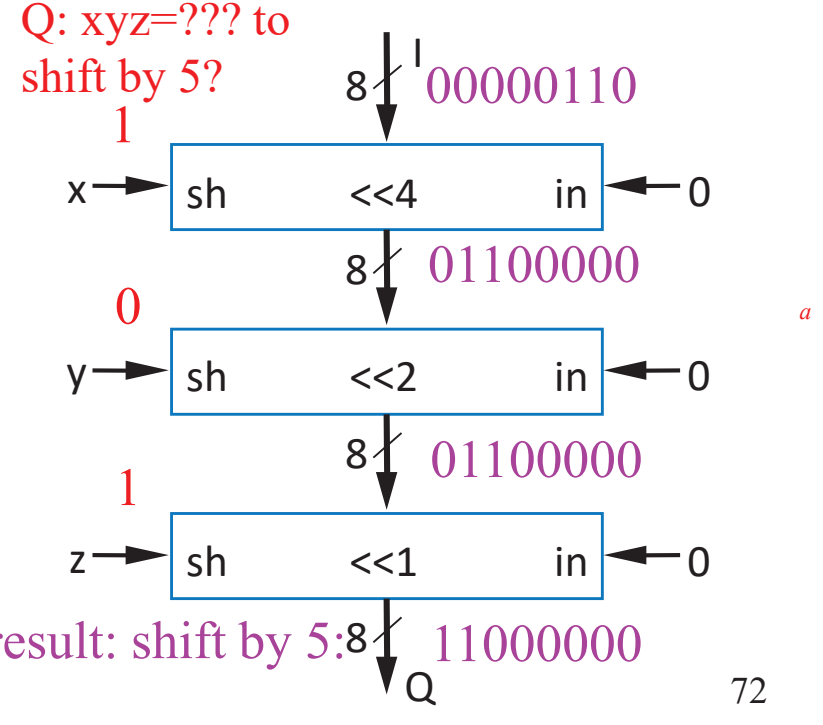
Copyright © 2010
Frank Vahid

Barrel Shifter

- A shifter that can shift by any amount
  - 4-bit barrel left shift can shift left by 0, 1, 2, or 3 positions
  - 8-bit barrel left shifter can shift left by 0, 1, 2, 3, 4, 5, 6, or 7 positions
    - (Shifting an 8-bit number by 8 positions is pointless -- you just lose all the bits)
- Could design using 8x1 muxes
  - Too many wires
- More elegant design
  - Chain three shifters: 4, 2, and 1
  - Can achieve any shift of 0..7 by enabling the correct combination of those three shifters, i.e., shifts should sum to desired amount



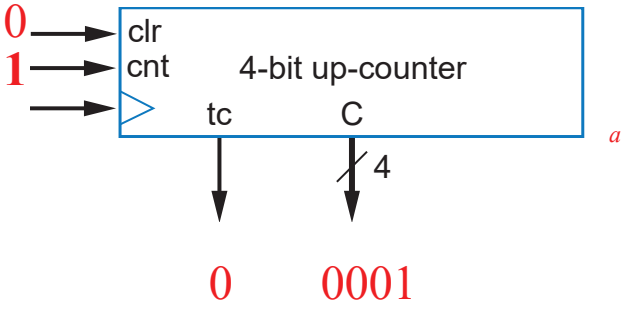
Shift by 1 shifter uses 2x1 muxes. 8x1 mux solution for 8-bit barrel shifter: too many wires.

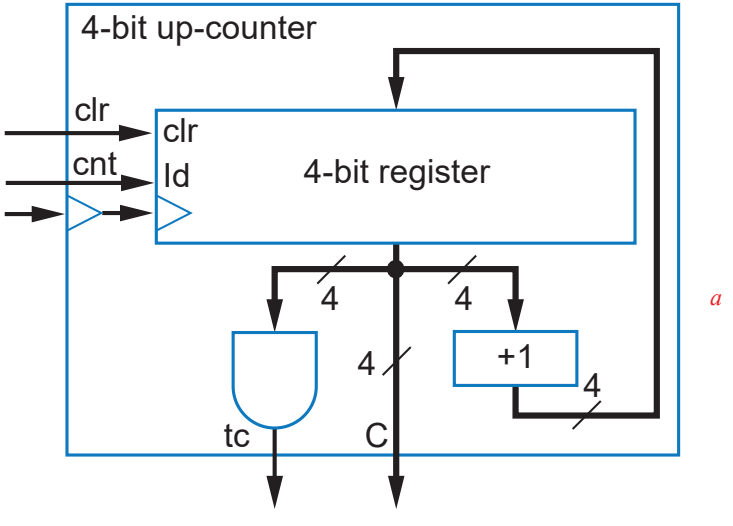


Digital Design 2e
Copyright © 2010
Frank Vahid

Counters and Timers

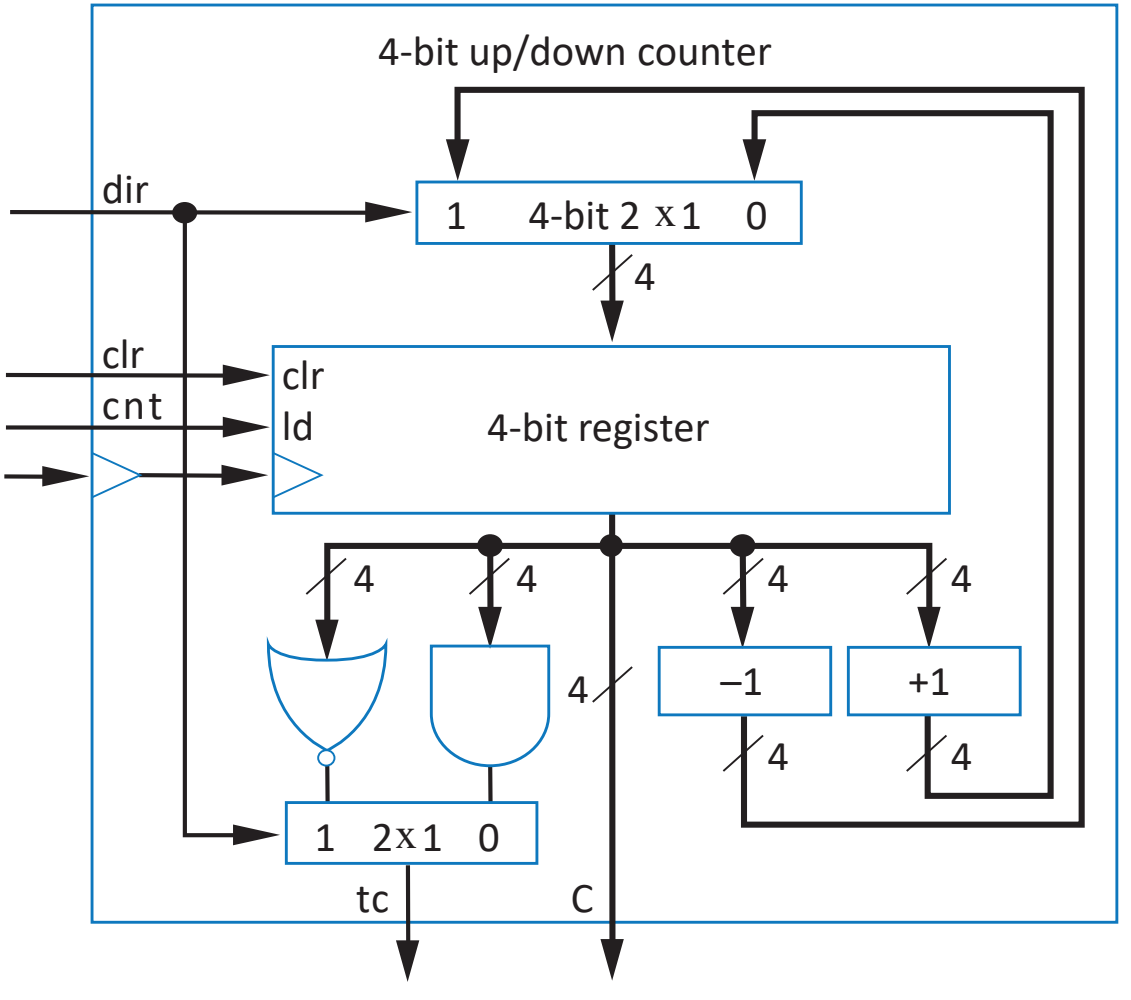
- ***N-bit up-counter***: N-bit register that can increment (add 1) to its own value on each clock cycle
  - 0000, 0001, 0010, 0011, ..., 1110, 1111, 0000
  - Count "rolls over" from 1111 to 0000
    - Terminal (last) count, tc, equals1 during value just before rollover
- Internal design
  - Register, incrementer, and N-input AND gate to detect terminal count





Up/Down-Counter

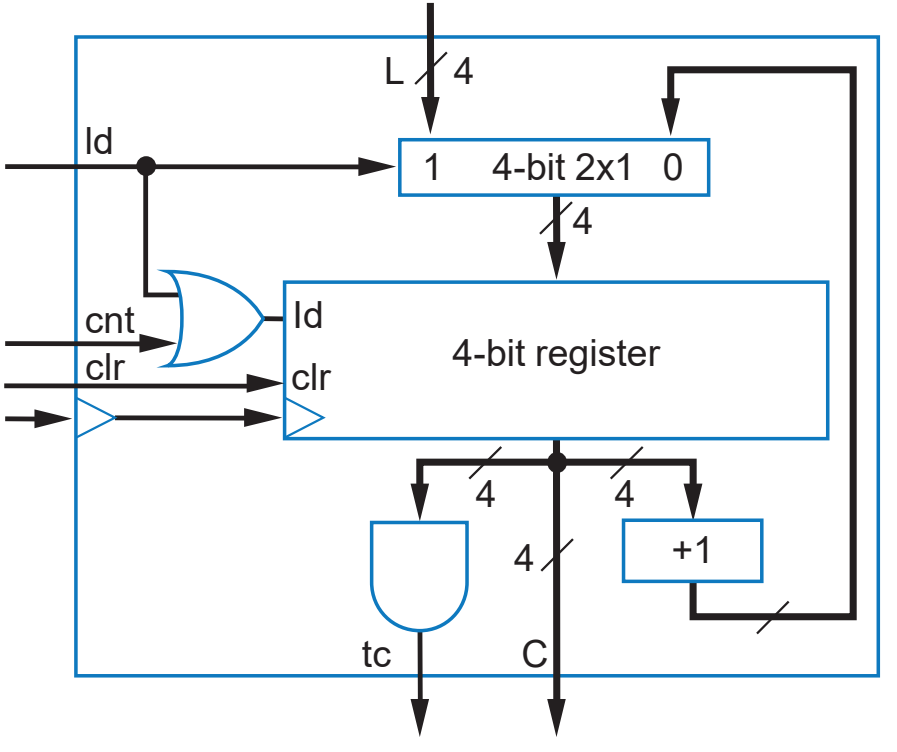
- Can count either up or down
  - Includes both incrementer and decrementer
  - Use dir input to select, via 2x1 mux: dir=0 means up
  - Likewise, dir selects appropriate terminal count value (all 1s or all 0s)



Digital Design 2e
Copyright © 2010
Frank Vahid

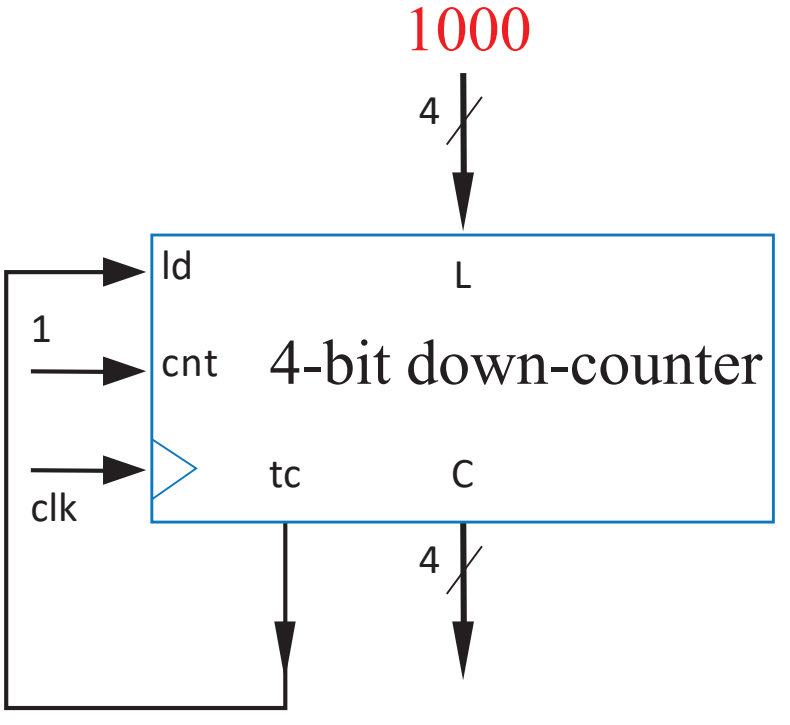
Counter with Load

- Up-counter that can be loaded with external value
  - Designed using 2x1 mux. ld input selects incremented value or external value
  - Load the internal register when loading external value or when counting
  - Note that ld has priority over cnt



Counter with Parallel Load

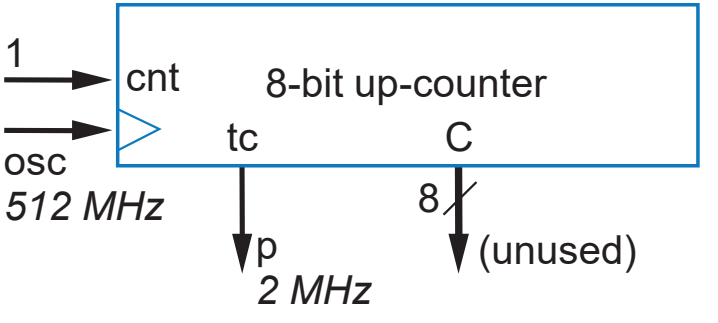
- Useful to create pulses at specific multiples of clock
  - Not just at N-bit counter's natural wrap-around of 2^N
- Example: Pulse every 9 clock cycles
  - Use 4-bit down-counter with parallel load
  - Set parallel load input to 8 (1000)
  - Use terminal count to reload
    - When count reaches 0, next cycle loads 8.
  - Why load 8 and not 9? Because 0 is included in count sequence:
    - 8, 7, 6, 5, 4, 3, 2, 1, 0 → 9 counts



Copyright © 2010
Frank Vahid

Common Counter Use: Clock Divider

- Suppose have 512 Hz oscillator, but want 2 Hz pulse
  - Thus, want divide fast clock by 256
  - Design using 8-bit up-counter, use tc output as pulse
    - Counts from 0 to 255 (256 counts), so pulses tc every 256 cycles



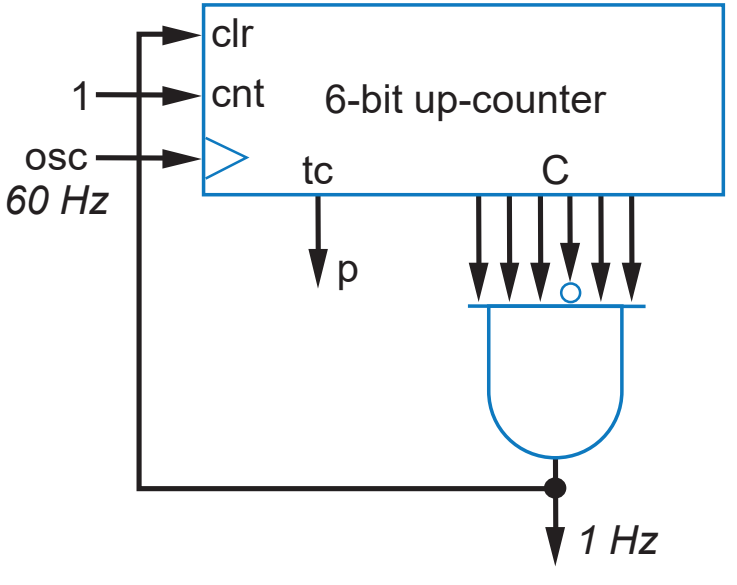


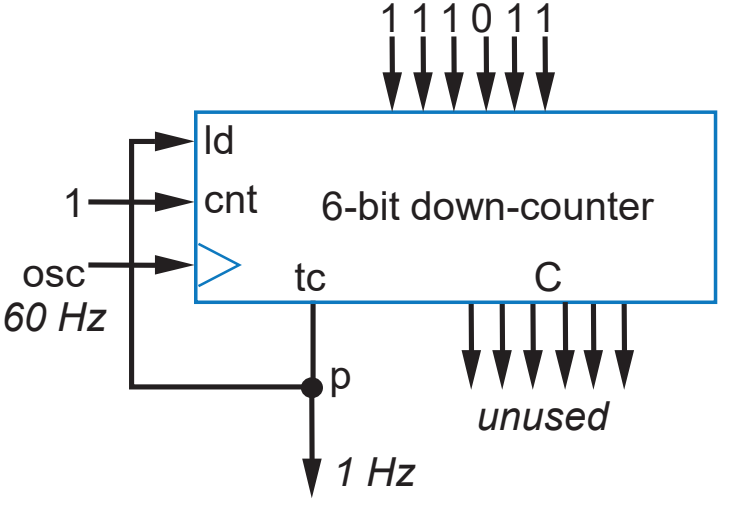
Digital Design 2e
Copyright © 2010
Frank Vahid

Clock Division by Amount not Power of 2

Example: 1 Hz Pulse Generator from 60 Hz Clock

- Example: U.S. electricity standard uses 60 Hz signal
  - Device may convert that to 1 Hz signal to count seconds
- Use 6-bit up-counter
  - Up counter: Detected count of 59 clears counter to 0; tc thus pulses every 60 cycles
    - Note: Detect 59, not 60 (because the first number, 0, is included)
  - Down counter approach also possible: When count reaches 0, load 59

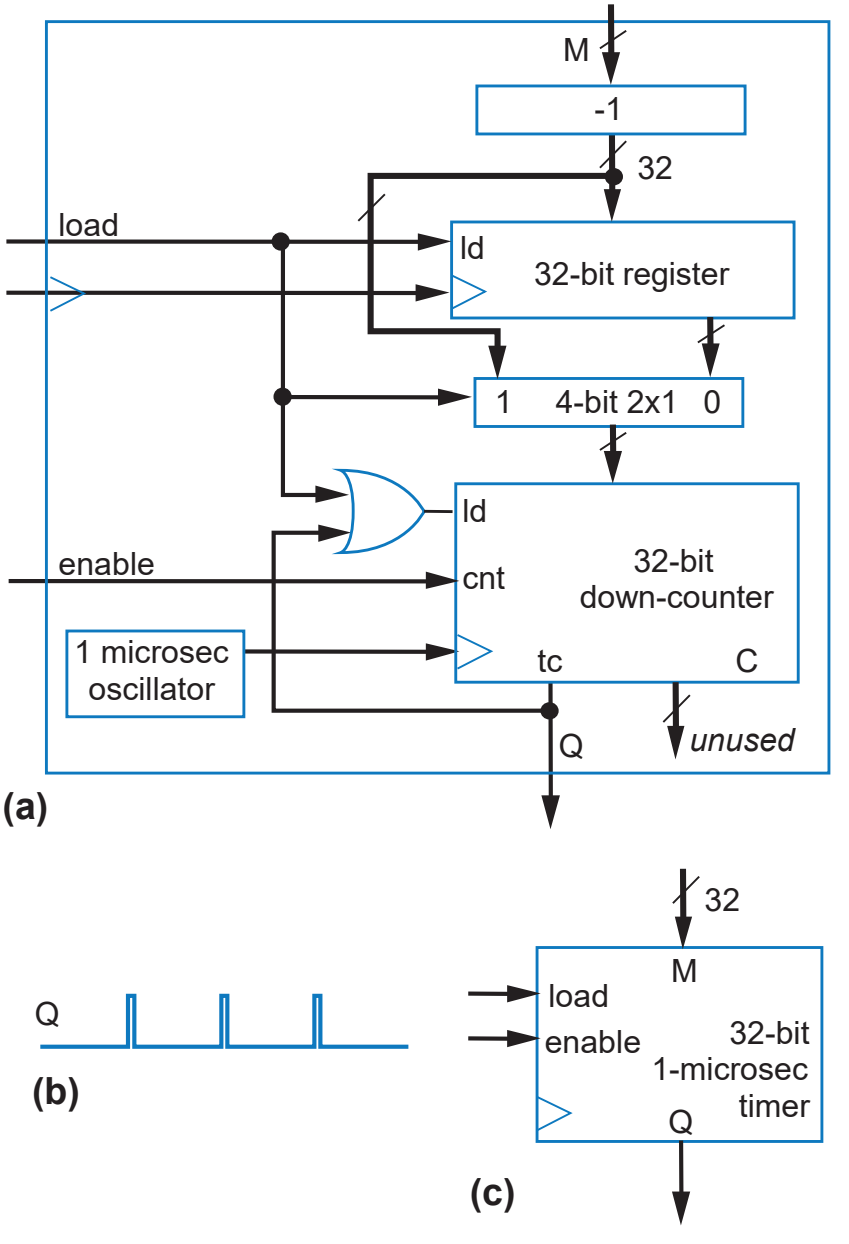




Copyright © 2010
Frank Vahid

Timers

- Pulses output at user-specified timer interval when enabled
  - "Ticks" like a clock
  - Interval specified as multiple of base time unit
  - If base is 1 microsec and user wants pulse every 300 ms, loads 300,000 into timer
- Can design using oscillator, register, and down-counter

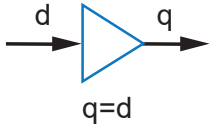


Digital Design 2e
Copyright © 2010
Frank Vahid

Register File

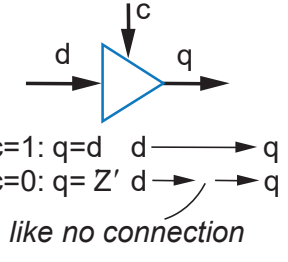
- Internal design uses drivers and bus

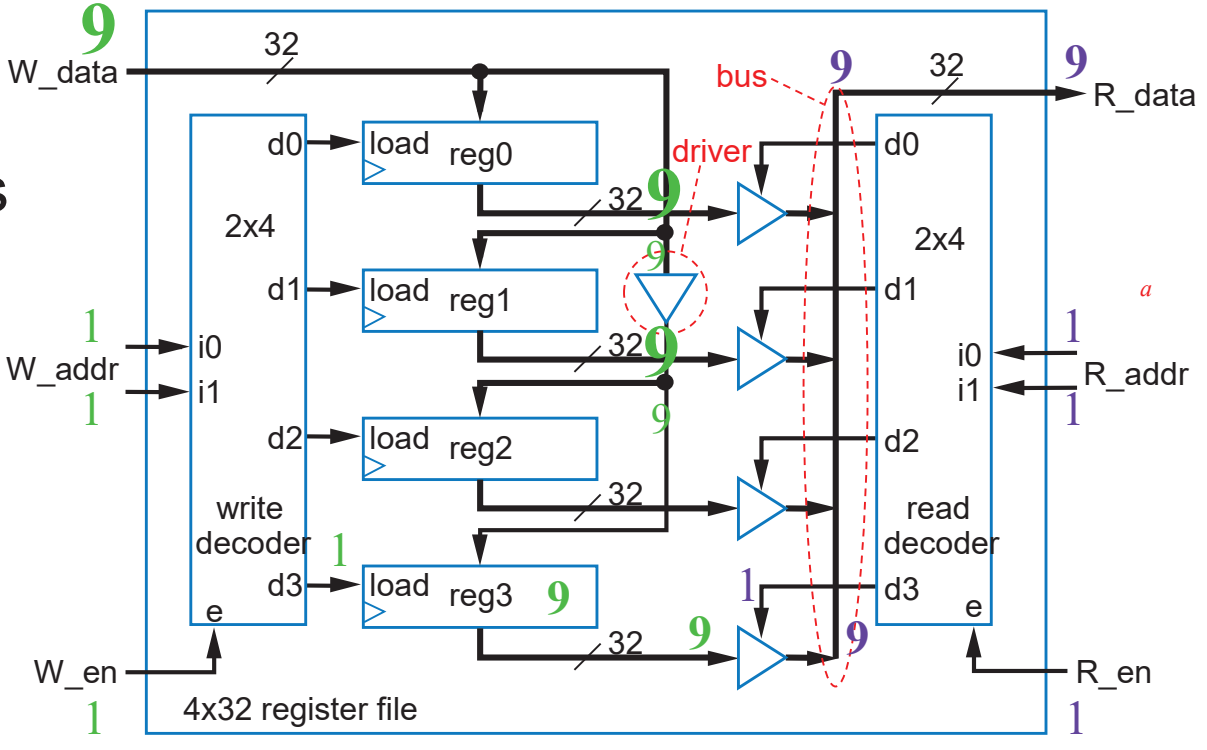
driver



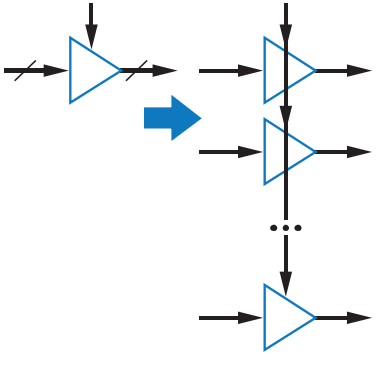
Boosts signal

three-state driver





Internal design of 4x32 RF; 16x32 RF follows similarly



Note: Each driver in figure actually represents 32 1-bit drivers

Digital Design 2e
Copyright © 2010
Frank Vahid